

JAVA INTERVIEW PREPARATION

CHAPTER 37

SPRING MVC REQUEST LIFECYCLE

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Spring MVC Request Lifecycle: Filters, DispatcherServlet, Handler Mapping, and Response Writing

Senior / Lead Java Developer Reading Chapter

Spring MVC is a request-dispatching pipeline built on the Servlet API. Understanding where filters, interceptors, argument resolvers, controllers, validation, exception resolvers, message converters, and asynchronous processing run allows senior developers to place behavior at the correct boundary and diagnose slow or incorrect responses.

End-to-End Flow

```
client
-> servlet container filter chain
-> DispatcherServlet
-> HandlerMapping finds controller method
-> HandlerAdapter invokes argument resolvers
-> controller/service
-> return-value handlers and message converters
-> response

exception -> HandlerExceptionResolver chain
```

Spring Security normally operates in the Servlet filter chain before DispatcherServlet. MVC interceptors run later around handler execution and cannot replace security filters for the complete web boundary.

DispatcherServlet

DispatcherServlet is the front controller. It coordinates strategies obtained from its WebApplicationContext rather than containing application logic itself.

It asks HandlerMapping for a handler, chooses a HandlerAdapter that can invoke it, processes the result through return-value handling or view resolution, and delegates exceptions to resolvers.

```
@RestController
@RequestMapping("/orders")
class OrderController {
    @GetMapping("/{id}")
    OrderResponse get(@PathVariable UUID id) {
        return service.get(id);
    }
}
```

Handler Mapping

RequestMappingHandlerMapping builds mappings from path, HTTP method, parameters, headers, consumes, and produces conditions.

```
@PostMapping(
    consumes = MediaType.APPLICATION_JSON_VALUE,
    produces = MediaType.APPLICATION_JSON_VALUE
)
ResponseBody<OrderResponse> create(@Valid @RequestBody CreateOrder request) { }
```

Ambiguous mappings should fail clearly at startup. Keep routes predictable; overloading behavior by subtle header or parameter combinations makes operations and documentation difficult.

Argument Resolution

HandlerMethodArgumentResolver converts request context into controller arguments: path variables, query parameters, headers, request bodies, principals, pageable objects, and custom types.

```
HTTP request -> argument resolvers -> typed controller parameters
```

Custom argument resolvers can provide tenant or correlation context, but avoid hidden database queries or authorization decisions that developers cannot see from the method signature. Security should remain explicit and auditable.

Conversion of simple values uses conversion services, while request-body deserialization uses `HttpMessageConverters`. These are different extension points.

Message Conversion and Content Negotiation

For `@ResponseBody` or `RestController` methods, a return-value handler selects an `HttpMessageConverter` based on return type, requested media types, and supported formats.

```
Java response object -> Jackson converter -> JSON bytes
```

A Java field existing does not guarantee it appears in JSON; visibility, records, annotations, modules, and `ObjectMapper` configuration matter. Conversely, accepting a field in a persistence entity does not mean it belongs in the external contract. Use explicit API DTOs.

Content negotiation should favor stable media types and explicit `Accept/Content-Type`. URL suffix and parameter negotiation can create caching and security surprises when enabled casually.

Filters Versus Interceptors

Servlet filters can wrap the entire request and response, including security, compression, CORS, and requests that never map to a controller. `HandlerInterceptor` operates around mapped MVC handlers.

```
filter:      servlet boundary, before DispatcherServlet
interceptor: MVC handler boundary
controller advice: controller binding and exception concerns
AOP:         Spring bean method boundary
```

Choose the layer from the concern. Correlation IDs often begin in a filter so error responses and security failures also receive them.

Controller Responsibilities

Controllers translate transport to application commands and translate application results to HTTP. They should not own transaction orchestration, complex business rules, or persistence entities.

```
@PostMapping
ResponseBody<OrderResponse> create(@Valid @RequestBody CreateOrderRequest request) {
    OrderId id = service.create(mapper.toCommand(request));
    URI location = linkTo(methodOn(OrderController.class).get(id.value())).toUri();
    return ResponseEntity.created(location).body(service.get(id));
}
```

Thin does not mean no logic. HTTP status, headers, conditional requests, pagination, and media types are controller concerns.

Validation Flow

`@Valid` or `@Validated` triggers Bean Validation where configured. Request-body validation failures and method-parameter validation can produce different exception types depending on framework version and signature.

Validation has layers:

```
deserialization -> structural validation -> application/domain invariant -> database constraint
```

Annotations such as `@NotBlank` protect shape. Domain rules such as credit eligibility belong in application or domain services. Database uniqueness remains the final race-safe authority even if the API checks early.

Exception Resolution

Exceptions flow through `HandlerExceptionResolver` implementations. `@ExceptionHandler` methods and `@ControllerAdvice` provide application mappings, while framework resolvers handle standard MVC exceptions and status annotations.

Returning a response directly from a filter bypasses MVC exception handling unless the filter delegates appropriately. Place global error mapping with awareness of the layer producing the error.

Synchronous and Asynchronous MVC

A normal controller occupies a servlet request thread while executing. Spring MVC can release the container thread when returning types such as `Callable`, `DeferredResult`, or supported asynchronous structures, then dispatch completion later.

```
request thread -> start async work -> release container thread  
async completes -> container dispatch -> write response
```

Async request handling does not make blocking work non-blocking; it moves it to another executor. That executor needs capacity, context propagation, timeouts, rejection, and shutdown.

Virtual-thread-enabled servlet execution can make synchronous blocking code scale differently, but downstream limits and transaction/context rules remain.

Streaming and Backpressure

`StreamingResponseBody` can write incrementally, but slow clients hold resources and require timeout and executor design. Servlet MVC streaming does not automatically provide end-to-end reactive backpressure.

For large exports, avoid building the complete byte array in heap. Stream bounded chunks, handle client disconnects, and ensure database cursors or files close.

Multipart Requests

Multipart upload limits must exist at server, framework, proxy, and application levels. A controller-level size check occurs after infrastructure may have buffered data.

Store uploads outside the web root, validate content rather than trusting filenames or media types, generate safe names, scan when required, and avoid path traversal. Streaming and temporary-file cleanup are operational concerns.

Request Context and ThreadLocal

Locale, security context, tracing, logging MDC, and request attributes often rely on current-thread context. Starting arbitrary asynchronous tasks loses or leaks this context unless supported propagation is used.

Do not pass `HttpServletRequest` deep into business or background services. Extract immutable required values at the boundary.

Production Diagnosis

For 404, ask whether no handler matched, a static-resource handler handled the path, security hid it, or a reverse proxy rewrote it. For 415, inspect request Content-Type and converters. For 406, inspect Accept and supported response media types. For 400, separate malformed JSON, conversion, and validation failures.

Measure filter time, handler time, serialization time, response bytes, executor queue wait, and client disconnects. A slow API may finish business work quickly but spend time serializing a huge graph.

The Detailed Dispatch Sequence

The compact flow expands into several strategy decisions:

```
filter chain
-> DispatcherServlet obtains mapped handler + interceptors
-> HandlerAdapter prepares invocation
-> interceptors preHandle in registration order
-> argument resolvers obtain each parameter
-> data binding, conversion, deserialization, validation
-> controller method invocation
-> return-value handler chooses response strategy
-> interceptors postHandle in reverse order when applicable
-> message conversion or view rendering
-> interceptors afterCompletion in reverse order
-> filter chain unwinds
```

`preHandle` can stop normal handler execution by returning false, but the interceptor then owns any response it intended to produce. `postHandle` is less useful for `@ResponseBody` methods once return processing is underway and is not a universal error hook. `afterCompletion` receives final completion for cleanup and timing when dispatch reaches that interceptor.

Do not assume one physical pass. Forward, error, and asynchronous dispatch can re-enter parts of the servlet pipeline. A filter should define which dispatcher types it applies to and use a once-per-request abstraction appropriately when duplicate execution would corrupt metrics or authentication state.

HandlerAdapter Is the Invocation Engine

`DispatcherServlet` does not directly understand every controller style. A `HandlerAdapter` knows how to invoke the chosen handler. For annotated controllers, the adapter coordinates argument resolvers, binding, validation, method invocation, and return-value handlers.

```
mapped HandlerMethod
+-- parameter 1 -> PathVariable resolver
+-- parameter 2 -> RequestBody resolver -> converter -> validation
+-- parameter 3 -> Principal resolver
+-- return value -> ResponseEntity handler -> converter
```

Resolver order matters because the first resolver that supports a parameter normally handles it. A custom resolver with an overly broad `supportsParameter` can steal standard parameters. Keep custom marker annotations narrow and test coexistence with built-in behavior.

Return-value handling is also type-driven. `ResponseEntity`, a body object, a view name, a model, an asynchronous type, and streaming output have different handlers. Returning `null` can therefore mean no body, no view, or another handler-specific outcome; use explicit response contracts for APIs.

Data Binding Is a Security Boundary

Binding request parameters directly onto a rich domain or persistence object can expose fields the API never intended clients to control. This is mass assignment.

```
incoming JSON contains: role=ADMIN, accountStatus=APPROVED
entity exposes setters
generic binding copies values
authorization invariant bypassed
```

Use request DTOs containing only writable API fields, then map to an application command. Jackson visibility and constructor selection, MVC data binding, and Bean Validation serve different jobs; none replaces authorization in the application service.

Limit collection sizes, nesting depth, string length, and numeric ranges. A syntactically valid JSON document can still consume excessive CPU or memory during parsing and validation. Server/proxy body limits act before controller validation and should align with endpoint contracts.

Content Negotiation as a Selection Algorithm

The request `Content-Type` describes the representation being sent. The `Accept` header describes representations acceptable in the response.

```
request body + Content-Type
-> find converter that can read target Java type
-> otherwise 415 Unsupported Media Type

controller return + Accept/produces
-> find compatible response media type and writer
-> otherwise 406 Not Acceptable
```

Malformed bytes or a JSON/type mismatch generally fail during reading and often become a 400-class response, which is different from having no reader for the media type. Record negotiated media type and selected converter in targeted diagnostics rather than returning one generic "bad request" message for every case.

Version APIs through explicit media types, paths, or another deliberate contract. Do not allow converter discovery order to become accidental version selection. Set `Vary: Accept` where representations differ so caches do not serve one media type for another request.

Exception Resolution and Problem Details

The resolver chain attempts to translate an exception into a handled response. Local `@ExceptionHandler` methods, applicable controller advice, response-status mappings, and framework resolvers can all participate according to their order and match.

```
controller throws DomainNotFound
-> resolver searches local handler
-> searches applicable advice
-> maps to 404 ProblemDetail
-> response body written by converter
```

Use a stable error schema such as problem details with a machine-readable type/code, safe title/detail, status, instance or correlation identifier, and field violations where appropriate. Do not expose stack traces,

SQL, internal class names, or secret configuration.

Exception-handler selection should use specific types. A global `ExceptionHandler` handler can hide framework status exceptions or programming defects if ordered carelessly. Log once at the layer that owns the failure; controller advice should not duplicate logs already emitted by the application or container.

Once the HTTP response is committed, headers and status can no longer be replaced reliably. A failure during streaming may only allow connection termination and server logging, not a clean JSON error envelope.

Security Filters and MVC Authorization

Spring Security operates primarily through a servlet filter chain, allowing it to protect requests before handler mapping and to cover endpoints outside annotated controllers. Method security can add a bean-method policy deeper in the call graph.

```
security filter: authenticate request and apply web authorization
controller: translate authenticated request to command
method security: protect application capability when called from any adapter
```

MVC interceptors are not a substitute for the security filter chain. An interceptor only sees mapped handlers and can be bypassed by error dispatch, static resources, or other servlet endpoints depending on configuration. Avoid using route presence to infer authorization; express security rules directly and test denial paths.

Async MVC Has a Second Dispatch

When a controller returns `Callable` or `DeferredResult`, MVC starts asynchronous servlet processing and releases the original container thread. Completion later triggers another dispatch to finish return-value handling.

```
dispatch 1: filter -> MVC -> start async -> original thread exits
worker: execute application work -> complete result or failure
dispatch 2: MVC resumes -> converter/error resolver -> response
```

Timeout can occur at servlet, MVC async, executor queue, application, or downstream-client layers. Define one deadline strategy and cancel underlying work where meaningful. A timed-out HTTP response does not prove the queued JDBC or remote call stopped.

Interceptors and filters may receive async-specific callbacks or multiple dispatches. Logging MDC, security context, locale, and observation context need supported propagation to the worker and cleanup afterward. Measure original request time, executor queue wait, worker execution, redispach, and response writing separately.

Streaming, Commitment, and Slow Clients

Streaming changes error and resource semantics. After the first bytes flush, the status and headers are committed. If row 50,000 fails, the server cannot turn the partially downloaded CSV into a normal 500 JSON response.

```
open cursor/file
-> write bounded chunk
-> flush according to policy
-> client reads slowly or disconnects
-> detect failure and close every resource
```

Apply read/write/idle timeouts at container and proxy layers. Limit concurrent exports and use a dedicated executor or permit boundary so slow clients do not consume all request capacity. Prefer a durable

asynchronous export job with object storage when generation is long, restartability matters, or clients may disconnect.

Servlet output is push-oriented and does not provide the same end-to-end demand signaling as a reactive pipeline. Buffer sizes and flushing can create memory amplification even when the source is streamed.

Forwarded Headers and External Request Identity

Behind a reverse proxy, the servlet request may see internal scheme, host, port, and client address. Trusted forwarded-header processing can reconstruct external links and security decisions, but accepting spoofed forwarding headers from untrusted clients is dangerous.

```
external https://api.example/orders
-> proxy forwards to http://app:8080/orders
-> trusted forwarding metadata reconstructs external URI
```

Configure proxy trust and header handling consistently at the platform and application. Test redirects, generated links, secure-cookie behavior, client-IP logging, and rate limiting through the real ingress path.

Testing Each Layer Deliberately

controller unit test	mapping-independent translation logic
MockMvc focused test	mapping, binding, validation, advice, security integration
full server test	filters, container, serialization, async, real HTTP behavior
end-to-end ingress test	proxy headers, limits, TLS, compression, routing

Mock MVC testing is fast and valuable but does not reproduce every servlet-container or network behavior. For streaming, multipart limits, asynchronous timeouts, compression, and forwarded headers, include tests with a running server and representative infrastructure.

Contract tests should assert status, headers, media type, body schema, error codes, and validation paths. Avoid assertions that couple consumers to internal exception class names.

Request Observability Without Cardinality Explosion

Record route templates such as `/orders/{id}`, not raw paths containing customer IDs. Useful dimensions include HTTP method, route, status class, exception category, negotiated media type, and outcome. Keep correlation IDs in traces/logs rather than metric labels.

Separate timings:

```
filter/security
handler mapping and argument binding
application service
serialization
async queue/execution
response write or client backpressure
```

An overall timer is necessary but insufficient. The breakdown identifies whether an apparently slow controller is actually waiting for authentication, parsing a huge request, queueing async work, or writing to a slow client.

A Strong Interview Explanation

Spring MVC is a Servlet-based front-controller pipeline. Filters surround the servlet boundary; `DispatcherServlet` uses a handler mapping to find a handler and interceptors, a handler adapter to resolve arguments and invoke it, and return-value handlers plus message converters or view resolvers to create the response. Exceptions go through ordered exception resolvers.

A senior answer distinguishes **Content-Type** reading failures from **Accept** writing failures, structural validation from domain authorization, and filters from interceptors and AOP. Async MVC releases the original request thread but moves work to an executor and later redispaches; it still needs deadlines, context propagation, and downstream bounds. Streaming avoids full buffering but commits the response early and makes slow-client and partial-failure policy explicit.

Interview-Ready Summary

Filters surround the Servlet boundary. DispatcherServlet selects a mapped handler and adapter, argument resolvers build method arguments, the controller invokes application behavior, and return-value handlers plus converters create the response. Exceptions go through resolver chains.

Use filters for protocol-wide concerns, interceptors for MVC handler concerns, advice for controller exceptions, and AOP for bean methods. Async MVC releases the request thread but moves work to another execution policy; it does not remove blocking or capacity limits.

Revision Notes

- DispatcherServlet is Spring MVC's front controller.
- HandlerMapping chooses a handler from request conditions.
- HandlerAdapter invokes the selected handler.
- Argument resolvers build typed controller parameters.
- ConversionService and HttpMessageConverter serve different conversions.
- Message converters serialize and deserialize request/response bodies.
- RestController implies response-body handling.
- Filters run outside and before the MVC handler pipeline.
- Interceptors wrap mapped controller execution.
- Controllers translate between HTTP and application contracts.
- Explicit DTOs protect the API from persistence-model leakage.
- Structural validation differs from domain and database invariants.
- Exception resolvers map controller-pipeline failures.
- Filter failures do not automatically enter MVC advice.
- Async MVC releases the container thread and uses another executor.
- Async execution still needs timeout, context, queue, and rejection policy.
- Streaming avoids full buffering but creates slow-client resource concerns.
- Multipart limits must exist before controller processing.
- Extract immutable request context before background work.
- Diagnose 400, 404, 406, and 415 at their distinct pipeline stages.
- Interceptor preHandle, postHandle, and afterCompletion have different guarantees.
- Custom argument resolvers should match narrowly and avoid hidden I/O.
- Bind requests to dedicated DTOs to prevent mass assignment.
- Content-Type selects request readers; Accept participates in response selection.
- Problem responses need stable safe fields and specific exception mapping.
- Security filters cover a broader boundary than MVC interceptors.
- Async MVC normally completes through a later servlet dispatch.
- Async timeout does not guarantee underlying work is cancelled.
- Streaming commits headers early and needs partial-failure policy.
- Forwarded headers must be trusted only from configured proxy boundaries.
- Use route templates, not raw paths, as metric dimensions.
- Test container and ingress behavior where MockMvc cannot reproduce it.